📘 SQL Interview Questions with Queries

1. Write a query to find the second highest salary.

```
SELECT MAX(salary) AS SecondHighestSalary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

2. Write a query to find the Nth highest salary.

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET N-1;
```

3. Write a query to find employees earning more than their manager.

```
SELECT e.name AS Employee, e.salary, m.name AS Manager, m.salary AS
ManagerSalary
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```


4. Write a query to find employees with the same salary as their manager.

```
SELECT e.name AS Employee, e.salary, m.name AS Manager
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary = m.salary;
```

5. Write a query to find employees earning above the company average salary.

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

6. Write a query to find duplicate records.

```sql
SELECT name, COUNT(*)
FROM employees
GROUP BY name
HAVING COUNT(*) > 1;
```

7. Write a query to count employees per department having more than 5 employees.

```sql
SELECT department_id, COUNT(*) AS num_employees
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

8. Write a query to perform conditional aggregation (male/female count).

```sql
SELECT department_id,
COUNT(CASE WHEN gender = 'M' THEN 1 END) AS male_count,
COUNT(CASE WHEN gender = 'F' THEN 1 END) AS female_count
FROM employees
GROUP BY department_id;
```


9. Write a query to count employees per job title.

```sql
SELECT job_title, COUNT(*) AS num_employees
FROM employees
GROUP BY job_title;
```

10. Write a query to calculate running total of salaries by department.

```sql
SELECT name, department_id, salary,
SUM(salary) OVER (PARTITION BY department_id ORDER BY id) AS running_total
FROM employees;
```

11. Write a query to calculate salary difference using LAG.

```
SELECT name, salary,
salary - LAG(salary) OVER (ORDER BY id) AS salary_diff
FROM employees;
```

12. Write a query to rank employees by salary.

```
SELECT name, salary,
RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

13. Write a query to list top 5 highest-paid employees per department.

```
SELECT *
FROM (
  SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary
DESC) AS rn
  FROM employees e
) sub
WHERE rn <= 5;
```

14. Write a recursive query to find the full reporting chain.

```
WITH RECURSIVE reporting_chain AS (
  SELECT id, name, manager_id, 1 AS level
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, rc.level + 1
  FROM employees e
  JOIN reporting_chain rc ON e.manager_id = rc.id
)
SELECT * FROM reporting_chain ORDER BY level, id;
```

15. Write a recursive query to detect circular references in hierarchy.

```sql
WITH RECURSIVE mgr_path (id, manager_id, path) AS (
  SELECT id, manager_id, ARRAY[id]
  FROM employees
  WHERE manager_id IS NOT NULL
  UNION ALL
  SELECT e.id, e.manager_id, path || e.id
  FROM employees e
  JOIN mgr_path mp ON e.manager_id = mp.id
  WHERE NOT e.id = ANY(path)
)
SELECT DISTINCT id
FROM mgr_path
WHERE id = ANY(path);
```

16. Write a query to calculate running total of sales per customer.

```sql
SELECT customer_id, sale_date, amount,
SUM(amount) OVER (PARTITION BY customer_id ORDER BY sale_date) AS running_total
FROM sales;
```

17. Write a query to calculate percentage change in monthly sales.

```sql
SELECT product_id, sale_month, total_sales,
(total_sales - LAG(total_sales) OVER (PARTITION BY product_id ORDER BY
sale_month)) * 100.0 /
LAG(total_sales) OVER (PARTITION BY product_id ORDER BY sale_month) AS
pct_change
FROM (
  SELECT product_id, DATE_TRUNC('month', sale_date) AS sale_month, SUM(amount)
AS total_sales
  FROM sales
  GROUP BY product_id, sale_month
) monthly_sales;
```

18. Write a query to find top 3 products with highest sales each month.

```sql
WITH monthly_product_sales AS (
  SELECT product_id, DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS
total_sales
  FROM sales
  GROUP BY product_id, month
),
ranked_sales AS (
  SELECT *, RANK() OVER (PARTITION BY month ORDER BY total_sales DESC) AS
sales_rank
  FROM monthly_product_sales
)
SELECT product_id, month, total_sales
FROM ranked_sales
WHERE sales_rank <= 3
ORDER BY month, sales_rank;
```

19. Write a query to find employees with salary greater than average salary in the company, ordered by salary descending.

```sql
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

20. Write a query to aggregate employee names in a department as JSON array.

```sql
SELECT department_id, JSON_AGG(name) AS employee_names
FROM employees
GROUP BY department_id;
```

21. Write a query to get the first and last purchase date for each customer.

```sql
SELECT customer_id,
MIN(purchase_date) AS first_purchase,
MAX(purchase_date) AS last_purchase
FROM sales
GROUP BY customer_id;
```

22. Write a query to find departments with the highest average salary.

```sql
WITH avg_salaries AS (
  SELECT department_id, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department_id
)
SELECT *
FROM avg_salaries
WHERE avg_salary = (SELECT MAX(avg_salary) FROM avg_salaries);
```

23. Write a query to find employees without a department assigned.

```sql
SELECT *
FROM employees
WHERE department_id IS NULL;
```

24. Write a query to calculate the difference in days between two dates in the same table.

```sql
SELECT id, EXTRACT(DAY FROM (end_date) - (start_date)) AS days_difference
FROM projects;
```

25. Write a query to calculate moving average of salaries over last 3 employees ordered by hire date.

```sql
SELECT name, hire_date, salary,
AVG(salary) OVER (ORDER BY hire_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
AS moving_avg_salary
FROM employees;
```

26. Write a query to find the most recent purchase per customer using window functions.

```sql
SELECT *
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY purchase_date
DESC) AS rn
  FROM sales
) sub
WHERE rn = 1;
```

27. Write a query to detect hierarchical depth of each employee.

```
WITH RECURSIVE employee_depth AS (
  SELECT id, name, manager_id, 1 AS depth
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, ed.depth + 1
  FROM employees e
  JOIN employee_depth ed ON e.manager_id = ed.id
)
SELECT * FROM employee_depth;
```

28. Write a query to perform a self-join to find pairs of employees in the same department.

```
SELECT e1.name AS Employee1, e2.name AS Employee2, e1.department_id
FROM employees e1
JOIN employees e2 ON e1.department_id = e2.department_id AND e1.id < e2.id;
```

29. Write a query to pivot rows into columns (simulate fixed values).

```
SELECT department_id,
SUM(CASE WHEN job_title = 'Manager' THEN 1 ELSE 0 END) AS Managers,
SUM(CASE WHEN job_title = 'Developer' THEN 1 ELSE 0 END) AS Developers,
SUM(CASE WHEN job_title = 'Tester' THEN 1 ELSE 0 END) AS Testers
FROM employees
GROUP BY department_id;
```

30. Write a query to find customers who made purchases in every category.

```
SELECT customer_id
FROM sales
GROUP BY customer_id
HAVING COUNT(DISTINCT category_id) = (SELECT COUNT(DISTINCT category_id) FROM
sales);
```

31. Write a query to identify employees who haven't received a raise in more than a year.

```
SELECT e.name
FROM employees e
JOIN salary_history sh ON e.id = sh.employee_id
GROUP BY e.id, e.name
HAVING MAX(sh.raise_date) < CURRENT_DATE - INTERVAL '1 year';
```

32. Write a query to rank salespeople by monthly sales, resetting rank every month.

```
SELECT salesperson_id, sale_month, total_sales,
RANK() OVER (PARTITION BY sale_month ORDER BY total_sales DESC) AS monthly_rank
FROM (
  SELECT salesperson_id, DATE_TRUNC('month', sale_date) AS sale_month,
SUM(amount) AS total_sales
  FROM sales
  GROUP BY salesperson_id, sale_month
) monthly_sales;
```

33. Write a query to calculate percentage change in sales compared to previous month.

```
SELECT product_id, sale_month, total_sales,
(total_sales - LAG(total_sales) OVER (PARTITION BY product_id ORDER BY
sale_month)) * 100.0 /
LAG(total_sales) OVER (PARTITION BY product_id ORDER BY sale_month) AS
pct_change
FROM (
  SELECT product_id, DATE_TRUNC('month', sale_date) AS sale_month, SUM(amount)
AS total_sales
  FROM sales
  GROUP BY product_id, sale_month
) monthly_sales;
```


34. Write a query to retrieve last 5 orders for each customer.

```
SELECT *
FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date
DESC) AS rn
  FROM orders o
) sub
WHERE rn <= 5;
```

35. Write a query to find employees with no salary changes in last 2 years.

```sql
SELECT e.*
FROM employees e
LEFT JOIN salary_history sh ON e.id = sh.employee_id AND sh.change_date >=
CURRENT_DATE - INTERVAL '2 years'
WHERE sh.employee_id IS NULL;
```

36. Write a query to find department with lowest average salary.

```sql
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
ORDER BY avg_salary
LIMIT 1;
```

37. Write a query to list employees whose names start and end with same letter.

```sql
SELECT *
FROM employees
WHERE LEFT(name, 1) = RIGHT(name, 1);
```

38. Write a query to detect circular references in employee-manager hierarchy.

```sql
WITH RECURSIVE mgr_path (id, manager_id, path) AS (
  SELECT id, manager_id, ARRAY[id]
  FROM employees
  WHERE manager_id IS NOT NULL
  UNION ALL
  SELECT e.id, e.manager_id, path || e.id
  FROM employees e
  JOIN mgr_path mp ON e.manager_id = mp.id
  WHERE NOT e.id = ANY(path)
)
SELECT DISTINCT id
FROM mgr_path
WHERE id = ANY(path);
```

39. Write a query to get running total of sales per customer.

```sql
SELECT customer_id, sale_date, amount,
SUM(amount) OVER (PARTITION BY customer_id ORDER BY sale_date) AS running_total
FROM sales;
```

40. Write a query to find department-wise 90th percentile salary.

```sql
SELECT department_id, salary,
PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY salary) OVER (PARTITION BY
department_id) AS pct_90_salary
FROM employees;
```

41. Write a query to find employees whose salary is a prime number.

```sql
WITH primes AS (
  SELECT generate_series(2, (SELECT MAX(salary) FROM employees)) AS num
  EXCEPT
  SELECT num FROM (
    SELECT num, UNNEST(ARRAY(
      SELECT generate_series(2, FLOOR(SQRT(num))) AS divisor
    )) AS divisor
    WHERE num % divisor = 0
  ) composite
)
SELECT *
FROM employees
WHERE salary IN (SELECT num FROM primes);
```


42. Write a query to find employees who worked in multiple departments.

```sql
SELECT employee_id
FROM employee_department_history
GROUP BY employee_id
HAVING COUNT(DISTINCT department_id) > 1;
```

43. Write a query to calculate difference between current and previous sales partitioned by product.

```
SELECT product_id, sale_date, amount,
amount - LAG(amount) OVER (PARTITION BY product_id ORDER BY sale_date) AS
sales_diff
FROM sales;
```

44. Write a query to find employees at lowest level in hierarchy (no subordinates).

```
SELECT *
FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employees sub WHERE sub.manager_id = e.id);
```

45. Write a query to find average order value per month and category.

```
SELECT DATE_TRUNC('month', order_date) AS order_month, category_id,
AVG(order_value) AS avg_order_value
FROM orders
GROUP BY order_month, category_id;
```

46. Write a query to create running count of employees joined each year.

```
SELECT join_year, COUNT(*) AS yearly_hires,
SUM(COUNT(*)) OVER (ORDER BY join_year) AS running_total_hires
FROM (
  SELECT EXTRACT(YEAR FROM hire_date) AS join_year
  FROM employees
) sub
GROUP BY join_year
ORDER BY join_year;
```

47. Write a query to rank employees by salary within their department and calculate percent rank.

```
SELECT name, department_id, salary,
RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS salary_rank,
PERCENT_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS
salary_percent_rank
FROM employees;
```

48. Write a query to find products that have never been sold.

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN sales s ON p.product_id = s.product_id
WHERE s.sale_id IS NULL;
```

49. Write a query to find consecutive days where sales were above a threshold.

```
WITH flagged_sales AS (
  SELECT sale_date, amount,
  CASE WHEN amount > 1000 THEN 1 ELSE 0 END AS flag
  FROM sales
),
groups AS (
  SELECT sale_date, amount, flag,
  sale_date - INTERVAL '1 day' * ROW_NUMBER() OVER (ORDER BY sale_date) AS grp
  FROM flagged_sales
  WHERE flag = 1
)
SELECT MIN(sale_date) AS start_date, MAX(sale_date) AS end_date, COUNT(*) AS
consecutive_days
FROM groups
GROUP BY grp
ORDER BY consecutive_days DESC;
```

50. Write a query to concatenate employee names in each department.

```
SELECT department_id, STRING_AGG(name, ',') AS employee_names
FROM employees
GROUP BY department_id;
```

51. Write a query to find employees whose salary is above their department's average but below company-wide average.

```
SELECT *
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department_id =
e.department_id)
AND salary < (SELECT AVG(salary) FROM employees);
```

52. Write a query to list customers who purchased all products in a specific category.

```sql
SELECT customer_id
FROM sales
WHERE category_id = 10
GROUP BY customer_id
HAVING COUNT(DISTINCT product_id) = (
  SELECT COUNT(DISTINCT product_id)
  FROM products
  WHERE category_id = 10
);
```

53. Write a query to find employees with no entries in salary history.

```sql
SELECT e.*
FROM employees e
LEFT JOIN salary_history sh ON e.id = sh.employee_id
WHERE sh.employee_id IS NULL;
```

54. Write a query to show department with highest number of employees.

```sql
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
ORDER BY employee_count DESC
LIMIT 1;
```


55. Write a recursive query to list all ancestors (managers) of a given employee.

```sql
WITH RECURSIVE ancestors AS (
  SELECT id, name, manager_id
  FROM employees
  WHERE id = 123
  UNION ALL
  SELECT e.id, e.name, e.manager_id
  FROM employees e
  JOIN ancestors a ON e.id = a.manager_id
)
SELECT * FROM ancestors WHERE id != 123;
```

56. Write a query to calculate median salary by department using window functions.

```
SELECT DISTINCT department_id,
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) OVER (PARTITION BY
department_id) AS median_salary
FROM employees;
```

57. Write a query to find first and last purchase date for each customer including non-buyers.

```
SELECT c.customer_id,
MIN(s.purchase_date) AS first_purchase,
MAX(s.purchase_date) AS last_purchase
FROM customers c
LEFT JOIN sales s ON c.customer_id = s.customer_id
GROUP BY c.customer_id;
```

58. Write a query to calculate percentage difference between each month's sales and previous month.

```
WITH monthly_sales AS (
  SELECT DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS total_sales
  FROM sales
  GROUP BY month
)
SELECT month, total_sales,
(total_sales - LAG(total_sales) OVER (ORDER BY month)) * 100.0 /
LAG(total_sales) OVER (ORDER BY month) AS pct_change
FROM monthly_sales;
```


59. Write a query to find employees with longest tenure in their department.

```
WITH tenure AS (
  SELECT *, RANK() OVER (PARTITION BY department_id ORDER BY hire_date ASC) AS
tenure_rank
  FROM employees
)
SELECT *
FROM tenure
WHERE tenure_rank = 1;
```

60. Write a query to generate sales growth percentage compared to same month last year.

```sql
WITH monthly_sales AS (
  SELECT DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS total_sales
  FROM sales
  GROUP BY month
)
SELECT ms1.month, ms1.total_sales,
((ms1.total_sales - ms2.total_sales) * 100.0 / ms2.total_sales) AS growth_pct
FROM monthly_sales ms1
LEFT JOIN monthly_sales ms2 ON ms1.month = ms2.month + INTERVAL '1 year';
```

61. Write a query to identify overlapping shifts for employees.

```sql
SELECT s1.employee_id, s1.shift_id AS shift1, s2.shift_id AS shift2
FROM shifts s1
JOIN shifts s2 ON s1.employee_id = s2.employee_id AND s1.shift_id <> s2.shift_id
WHERE s1.start_time < s2.end_time AND s1.end_time > s2.start_time;
```

62. Write a query to calculate total revenue per customer and rank them.

```sql
SELECT customer_id, SUM(amount) AS total_revenue,
RANK() OVER (ORDER BY SUM(amount) DESC) AS revenue_rank
FROM sales
GROUP BY customer_id;
```

63. Write a query to find employees who never received a promotion.

```sql
SELECT e.*
FROM employees e
LEFT JOIN promotions p ON e.id = p.employee_id
WHERE p.employee_id IS NULL;
```

64. Write a query to find top 3 products with highest sales amount each month.

```sql
WITH monthly_product_sales AS (
  SELECT product_id, DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS
total_sales
  FROM sales
  GROUP BY product_id, month
),
ranked_sales AS (
  SELECT *, RANK() OVER (PARTITION BY month ORDER BY total_sales DESC) AS
sales_rank
  FROM monthly_product_sales
)
SELECT product_id, month, total_sales
FROM ranked_sales
WHERE sales_rank <= 3
ORDER BY month, sales_rank;
```

65. Write a query to find customers who placed orders only in last 30 days.

```sql
SELECT DISTINCT customer_id
FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'
AND customer_id NOT IN (
  SELECT DISTINCT customer_id
  FROM orders
  WHERE order_date < CURRENT_DATE - INTERVAL '30 days'
);
```

66. Write a query to find products that have never been ordered.

```sql
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN orders o ON p.product_id = o.product_id
WHERE o.order_id IS NULL;
```

67. Write a query to calculate total sales amount and number of orders per customer in last year.

```sql
SELECT customer_id, COUNT(*) AS total_orders, SUM(amount) AS total_sales
FROM sales
WHERE sale_date >= CURRENT_DATE - INTERVAL '1 year'
GROUP BY customer_id;
```

68. Write a query to find employees who have managed more than 3 projects.

```
SELECT manager_id, COUNT(DISTINCT project_id) AS project_count
FROM projects
GROUP BY manager_id
HAVING COUNT(DISTINCT project_id) > 3;
```

69. Write a query to calculate the difference in days between each employee's hire date and their manager's hire date.

```
SELECT e.name AS employee, m.name AS manager,
EXTRACT(DAY FROM (e.hire_date) - (m.hire_date)) AS days_difference
FROM employees e
JOIN employees m ON e.manager_id = m.id;
```

70. Write a query to find the department with the highest average years of experience.

```
SELECT department_id, AVG(EXTRACT(year FROM CURRENT_DATE - hire_date)) AS
avg_experience_years
FROM employees
GROUP BY department_id
ORDER BY avg_experience_years DESC
LIMIT 1;
```


71. Write a query to identify employees with overlapping project assignments.

```
SELECT p1.employee_id, p1.project_id AS project1, p2.project_id AS project2
FROM project_assignments p1
JOIN project_assignments p2 ON p1.employee_id = p2.employee_id AND p1.project_id
<> p2.project_id
WHERE p1.start_date < p2.end_date AND p1.end_date > p2.start_date;
```

72. Write a query to find customers who made purchases in every month of the current year.

```
WITH months AS (SELECT generate_series(1, 12) AS month),
customer_months AS (
  SELECT customer_id, EXTRACT(MONTH FROM purchase_date) AS month
  FROM sales
  WHERE EXTRACT(YEAR FROM purchase_date) = EXTRACT(YEAR FROM CURRENT_DATE)
  GROUP BY customer_id, EXTRACT(MONTH FROM purchase_date)
)
SELECT customer_id
FROM customer_months
GROUP BY customer_id
HAVING COUNT(DISTINCT month) = 12;
```

73. Write a query to list employees who earn more than all their subordinates.

```
SELECT e.id, e.name, e.salary
FROM employees e
WHERE e.salary > ALL (SELECT salary FROM employees sub WHERE sub.manager_id =
e.id);
```

74. Write a query to get the product with the highest sales for each category.

```
WITH category_sales AS (
  SELECT category_id, product_id, SUM(amount) AS total_sales,
  RANK() OVER (PARTITION BY category_id ORDER BY SUM(amount) DESC) AS sales_rank
  FROM sales
  GROUP BY category_id, product_id
)
SELECT category_id, product_id, total_sales
FROM category_sales
WHERE sales_rank = 1;
```

75. Write a query to find customers who haven't ordered in the last 6 months.

```
SELECT c.customer_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id
HAVING MAX(o.order_date) < CURRENT_DATE - INTERVAL '6 months' OR
MAX(o.order_date) IS NULL;
```

76. Write a query to find maximum salary gap between employees in same department.

```sql
SELECT department_id, MAX(salary) - MIN(salary) AS salary_gap
FROM employees
GROUP BY department_id;
```

77. Write a recursive query to compute total budget under each manager.

```sql
WITH RECURSIVE manager_budget AS (
  SELECT id, manager_id, budget
  FROM departments
  UNION ALL
  SELECT d.id, d.manager_id, mb.budget
  FROM departments d
  JOIN manager_budget mb ON d.manager_id = mb.id
)
SELECT manager_id, SUM(budget) AS total_budget
FROM manager_budget
GROUP BY manager_id;
```

78. Write a query to detect gaps in invoice numbers.

```sql
WITH numbered_invoices AS (
  SELECT invoice_number, ROW_NUMBER() OVER (ORDER BY invoice_number) AS rn
  FROM invoices
)
SELECT invoice_number + 1 AS missing_invoice
FROM numbered_invoices ni
WHERE (invoice_number + 1) <> (
  SELECT invoice_number FROM numbered_invoices WHERE rn = ni.rn + 1
);
```


79. Write a query to rank employees by salary within department but restart rank every 10 employees.

```sql
WITH ranked_employees AS (
  SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary
DESC) AS rn
  FROM employees e
)
SELECT *, ((rn-1)/10)+1 AS rank_group
FROM ranked_employees;
```

80. Write a query to calculate moving median of daily sales over last 7 days per product.

```
WITH daily_sales AS (
  SELECT product_id, sale_date, SUM(amount) AS total_sales
  FROM sales
  GROUP BY product_id, sale_date
)
SELECT product_id, sale_date,
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total_sales)
OVER (PARTITION BY product_id ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND
CURRENT ROW) AS moving_median
FROM daily_sales;
```

81. Write a query to find customers who purchased both product A and product B.

```
SELECT customer_id
FROM sales
WHERE product_id IN ('A', 'B')
GROUP BY customer_id
HAVING COUNT(DISTINCT product_id) = 2;
```

82. Write a query to find employees who worked in more than 3 different departments.

```
SELECT employee_id
FROM employee_department_history
GROUP BY employee_id
HAVING COUNT(DISTINCT department_id) > 3;
```

83. Write a query to calculate percentage contribution of each product's sales to total monthly sales.

```
WITH monthly_sales AS (
  SELECT product_id, DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS
product_sales
  FROM sales
  GROUP BY product_id, month
),
total_monthly_sales AS (
  SELECT month, SUM(product_sales) AS total_sales
  FROM monthly_sales
  GROUP BY month
)
SELECT m.product_id, m.month,
(m.product_sales * 100.0 / t.total_sales) AS pct_contribution
FROM monthly_sales m
JOIN total_monthly_sales t ON m.month = t.month;
```

84. Write a query to identify gaps and islands in attendance records.

```
WITH attendance_groups AS (
  SELECT employee_id, attendance_date,
  attendance_date - INTERVAL '1 day' * ROW_NUMBER() OVER (PARTITION BY
employee_id ORDER BY attendance_date) AS grp
  FROM attendance
)
SELECT employee_id, MIN(attendance_date) AS start_date, MAX(attendance_date) AS
end_date, COUNT(*) AS consecutive_days
FROM attendance_groups
GROUP BY employee_id, grp
ORDER BY employee_id, start_date;
```


85. Write a recursive query to list all descendants of a manager.

```
WITH RECURSIVE descendants AS (
  SELECT id, name, manager_id
  FROM employees
  WHERE manager_id = 100
  UNION ALL
  SELECT e.id, e.name, e.manager_id
  FROM employees e
  INNER JOIN descendants d ON e.manager_id = d.id
)
SELECT * FROM descendants;
```

86. Write a query to calculate a 3-month moving average of monthly sales per product.

```
WITH monthly_sales AS (
  SELECT product_id, DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS
total_sales
  FROM sales
  GROUP BY product_id, month
)
SELECT product_id, month, total_sales,
AVG(total_sales) OVER (PARTITION BY product_id ORDER BY month ROWS BETWEEN 2
PRECEDING AND CURRENT ROW) AS moving_avg
FROM monthly_sales;
```

87. Write a query to find employees who have the same hire date as their managers.

```
SELECT e.name AS employee_name, m.name AS manager_name, e.hire_date
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.hire_date = m.hire_date;
```

88. Write a query to find products with increasing sales over last 3 months.

```
WITH monthly_sales AS (
  SELECT product_id, DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS
total_sales
  FROM sales
  GROUP BY product_id, month
),
ranked_sales AS (
  SELECT product_id, month, total_sales,
  ROW_NUMBER() OVER (PARTITION BY product_id ORDER BY month DESC) AS rn
  FROM monthly_sales
)
SELECT ms1.product_id
FROM ranked_sales ms1
JOIN ranked_sales ms2 ON ms1.product_id = ms2.product_id AND ms1.rn = 1 AND
ms2.rn = 2
JOIN ranked_sales ms3 ON ms1.product_id = ms3.product_id AND ms3.rn = 3
WHERE ms3.total_sales < ms2.total_sales AND ms2.total_sales < ms1.total_sales;
```


89. Write a query to get the Nth highest salary per department.

```
SELECT department_id, salary
FROM (
  SELECT department_id, salary, ROW_NUMBER() OVER (PARTITION BY department_id
ORDER BY salary DESC) AS rn
  FROM employees
) sub
WHERE rn = N;
```

90. Write a query to find customers who made purchases in every category available.

```
SELECT customer_id
FROM sales
GROUP BY customer_id
HAVING COUNT(DISTINCT category_id) = (SELECT COUNT(DISTINCT category_id) FROM
sales);
```

91. Write a query to calculate average tenure of employees by department.

```
SELECT department_id, AVG(DATE_PART('year', CURRENT_DATE - hire_date)) AS
avg_tenure_years
FROM employees
GROUP BY department_id;
```

92. Write a query to find customers who purchased more than once in the same day.

```
SELECT customer_id, purchase_date, COUNT(*) AS purchase_count
FROM sales
GROUP BY customer_id, purchase_date
HAVING COUNT(*) > 1;
```

93. Write a query to find customers who purchased all products in a category.

```
SELECT customer_id
FROM sales
WHERE category_id = 10
GROUP BY customer_id
HAVING COUNT(DISTINCT product_id) = (
  SELECT COUNT(DISTINCT product_id)
  FROM products
  WHERE category_id = 10
);
```

94. Write a query to calculate cumulative percentage of total sales per product.

```
SELECT product_id, sale_amount,
SUM(sale_amount) OVER (ORDER BY sale_amount DESC) * 100.0 / SUM(sale_amount)
OVER () AS cumulative_pct
FROM sales;
```

95. Write a query to calculate monthly sales growth compared to same month last year.

```
WITH monthly_sales AS (
  SELECT DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS total_sales
  FROM sales
  GROUP BY month
)
SELECT ms1.month, ms1.total_sales,
((ms1.total_sales - ms2.total_sales) * 100.0 / ms2.total_sales) AS growth_pct
FROM monthly_sales ms1
LEFT JOIN monthly_sales ms2 ON ms1.month = ms2.month + INTERVAL '1 year';
```

96. Write a query to calculate 3-month moving average of monthly sales per product.

```
WITH monthly_sales AS (
  SELECT product_id, DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS
total_sales
  FROM sales
  GROUP BY product_id, month
)
SELECT product_id, month, total_sales,
AVG(total_sales) OVER (PARTITION BY product_id ORDER BY month ROWS BETWEEN 2
PRECEDING AND CURRENT ROW) AS moving_avg
FROM monthly_sales;
```

97. Write a query to find employees who have worked in multiple departments over time.

```
SELECT employee_id
FROM employee_department_history
GROUP BY employee_id
HAVING COUNT(DISTINCT department_id) > 1;
```

98. Write a query to find employees who have worked in more than 3 different departments.

```
SELECT employee_id
FROM employee_department_history
GROUP BY employee_id
HAVING COUNT(DISTINCT department_id) > 3;
```

99. Write a query to find employees who earn more than the average salary across the company but less than the highest salary in their department.

```
SELECT *
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees)
AND salary < (SELECT MAX(salary) FROM employees WHERE department_id =
e.department_id);
```


100. Write a query to find employees who earn more than all their subordinates.

```
SELECT e.id, e.name, e.salary
FROM employees e
WHERE e.salary > ALL (SELECT salary FROM employees sub WHERE sub.manager_id =
e.id);
```

101. Write a query to find employees who have the longest tenure within their department.

```
WITH tenure AS (
  SELECT *, RANK() OVER (PARTITION BY department_id ORDER BY hire_date ASC) AS
tenure_rank
  FROM employees
)
SELECT *
FROM tenure
WHERE tenure_rank = 1;
```

102. Write a query to find employees who have never made a sale.

```
SELECT e.id, e.name
FROM employees e
LEFT JOIN sales s ON e.id = s.employee_id
WHERE s.sale_id IS NULL;
```

103. Write a query to find employees who have never received a promotion.

```
SELECT e.*
FROM employees e
LEFT JOIN promotions p ON e.id = p.employee_id
WHERE p.employee_id IS NULL;
```

104. Write a query to find employees with no salary changes in the last 2 years.

```
SELECT e.*
FROM employees e
LEFT JOIN salary_history sh ON e.id = sh.employee_id AND sh.change_date >=
CURRENT_DATE - INTERVAL '2 years'
WHERE sh.employee_id IS NULL;
```

105. Write a query to find employees who haven’t received a raise in more than a year.

```
SELECT e.name
FROM employees e
JOIN salary_history sh ON e.id = sh.employee_id
GROUP BY e.id, e.name
HAVING MAX(sh.raise_date) < CURRENT_DATE - INTERVAL '1 year';
```

106. Write a query to find employees who don’t have a department assigned.

```sql
SELECT *
FROM employees
WHERE department_id IS NULL;
```

107. Write a query to find customers who have not made any purchase.

```sql
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN sales s ON c.customer_id = s.customer_id
WHERE s.sale_id IS NULL;
```

108. Write a query to find customers who purchased all products in a specific category.

```sql
SELECT customer_id
FROM sales
WHERE category_id = 10
GROUP BY customer_id
HAVING COUNT(DISTINCT product_id) = (
  SELECT COUNT(DISTINCT product_id)
  FROM products
  WHERE category_id = 10
);
```


109. Write a query to find customers who placed orders only in the last 30 days.

```sql
SELECT DISTINCT customer_id
FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'
AND customer_id NOT IN (
  SELECT DISTINCT customer_id
  FROM orders
  WHERE order_date < CURRENT_DATE - INTERVAL '30 days'
);
```

110. Write a query to find customers who purchased in every category available.

```
SELECT customer_id
FROM sales
GROUP BY customer_id
HAVING COUNT(DISTINCT category_id) = (SELECT COUNT(DISTINCT category_id) FROM
sales);
```

111. Write a query to calculate total revenue for each customer and rank them.

```
SELECT customer_id, SUM(amount) AS total_revenue,
RANK() OVER (ORDER BY SUM(amount) DESC) AS revenue_rank
FROM sales
GROUP BY customer_id;
```

112. Write a query to calculate total sales amount and number of orders per customer in the last year.

```
SELECT customer_id, COUNT(*) AS total_orders, SUM(amount) AS total_sales
FROM sales
WHERE sale_date >= CURRENT_DATE - INTERVAL '1 year'
GROUP BY customer_id;
```

113. Write a query to calculate cumulative distribution (CDF) of salaries.

```
SELECT name, salary,
CUME_DIST() OVER (ORDER BY salary) AS salary_cdf
FROM employees;
```

114. Write a query to calculate percentage change in sales compared to previous month for each product.

```
SELECT product_id, sale_month, total_sales,
(total_sales - LAG(total_sales) OVER (PARTITION BY product_id ORDER BY
sale_month)) * 100.0 /
LAG(total_sales) OVER (PARTITION BY product_id ORDER BY sale_month) AS
pct_change
FROM (
  SELECT product_id, DATE_TRUNC('month', sale_date) AS sale_month, SUM(amount)
AS total_sales
  FROM sales
  GROUP BY product_id, sale_month
) monthly_sales;
```

115. Write a query to find employees who are at the lowest level in the hierarchy (no subordinates).

```
SELECT *
FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employees sub WHERE sub.manager_id = e.id);
```

116. Write a query to calculate hierarchical depth of each employee.

```
WITH RECURSIVE employee_depth AS (
  SELECT id, name, manager_id, 1 AS depth
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, ed.depth + 1
  FROM employees e
  JOIN employee_depth ed ON e.manager_id = ed.id
)
SELECT * FROM employee_depth;
```


117. Write a query to list all ancestors (managers) of a given employee.

```
WITH RECURSIVE ancestors AS (
  SELECT id, name, manager_id
  FROM employees
  WHERE id = 123
  UNION ALL
  SELECT e.id, e.name, e.manager_id
  FROM employees e
  JOIN ancestors a ON e.id = a.manager_id
)
SELECT * FROM ancestors WHERE id != 123;
```

118. Write a query to list all descendants of a manager.

```
WITH RECURSIVE descendants AS (
  SELECT id, name, manager_id
  FROM employees
  WHERE manager_id = 100
  UNION ALL
  SELECT e.id, e.name, e.manager_id
  FROM employees e
  INNER JOIN descendants d ON e.manager_id = d.id
)
SELECT * FROM descendants;
```

119. Write a query to find the most recent purchase per customer.

```
SELECT *
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY purchase_date
DESC) AS rn
  FROM sales
) sub
WHERE rn = 1;
```

120. Write a query to find the second most recent order date per customer.

```
SELECT customer_id, order_date
FROM (
  SELECT customer_id, order_date,
  ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS rn
  FROM orders
) sub
WHERE rn = 2;
```

121. Write a query to retrieve the last 5 orders for each customer.

```
SELECT *
FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date
DESC) AS rn
  FROM orders o
) sub
WHERE rn <= 5;
```

122. Write a query to calculate salary percentile (e.g., 90th) per department.

```
SELECT department_id, salary,
PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY salary) OVER (PARTITION BY
department_id) AS pct_90_salary
FROM employees;
```

123. Write a query to calculate difference between current row and previous row's salary.

```
SELECT name, salary,
salary - LAG(salary) OVER (ORDER BY id) AS salary_diff
FROM employees;
```

124. Write a query to calculate difference in days between employee's hire date and manager's hire date.

```
SELECT e.name AS employee, m.name AS manager,
EXTRACT(DAY FROM (e.hire_date) - (m.hire_date)) AS days_difference
FROM employees e
JOIN employees m ON e.manager_id = m.id;
```

125. Write a query to find employees whose salary is above their department's average but below the overall company average.

```sql
SELECT *
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department_id =
e.department_id)
AND salary < (SELECT AVG(salary) FROM employees);
```

126. Write a query to find employees with salary in the top 10% in their department.

```sql
SELECT *
FROM (
  SELECT e.*, NTILE(10) OVER (PARTITION BY department_id ORDER BY salary DESC)
AS decile
  FROM employees e
) sub
WHERE decile = 1;
```

127. Write a query to find employees whose names start and end with the same letter.

```sql
SELECT *
FROM employees
WHERE LEFT(name, 1) = RIGHT(name, 1);
```

128. Write a query to find employees who joined in the last 6 months.

```sql
SELECT *
FROM employees
WHERE join_date >= CURRENT_DATE - INTERVAL '6 months';
```

129. Write a query to find employees who earn more than their manager.

```sql
SELECT e.name AS Employee, e.salary, m.name AS Manager, m.salary AS
ManagerSalary
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

130. Write a query to find employees who earn the same salary as their manager.

```sql
SELECT e.name AS Employee, e.salary, m.name AS Manager
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary = m.salary;
```

131. Write a query to find employees who have worked in multiple departments.

```sql
SELECT employee_id
FROM employee_department_history
GROUP BY employee_id
HAVING COUNT(DISTINCT department_id) > 1;
```

132. Write a query to find the department with the highest average salary.

```sql
WITH avg_salaries AS (
  SELECT department_id, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department_id
)
SELECT *
FROM avg_salaries
WHERE avg_salary = (SELECT MAX(avg_salary) FROM avg_salaries);
```


133. Write a query to find the department with the lowest average salary.

```sql
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
ORDER BY avg_salary
LIMIT 1;
```

134. Write a query to find the department with the highest number of employees.

```sql
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
ORDER BY employee_count DESC
LIMIT 1;
```

135. Write a query to find the maximum salary gap between employees in the same department.

```
SELECT department_id, MAX(salary) - MIN(salary) AS salary_gap
FROM employees
GROUP BY department_id;
```

136. Write a query to calculate average years of experience by department.

```
SELECT department_id, AVG(EXTRACT(year FROM CURRENT_DATE - hire_date)) AS
avg_experience_years
FROM employees
GROUP BY department_id
ORDER BY avg_experience_years DESC;
```

137. Write a query to find employees who joined in each year and calculate running total hires.

```
SELECT join_year, COUNT(*) AS yearly_hires,
SUM(COUNT(*)) OVER (ORDER BY join_year) AS running_total_hires
FROM (
  SELECT EXTRACT(YEAR FROM hire_date) AS join_year
  FROM employees
) sub
GROUP BY join_year
ORDER BY join_year;
```


138. Write a query to find first and last purchase date for each customer.

```
SELECT customer_id,
MIN(purchase_date) AS first_purchase,
MAX(purchase_date) AS last_purchase
FROM sales
GROUP BY customer_id;
```

139. Write a query to find first and last purchase date including customers who never purchased.

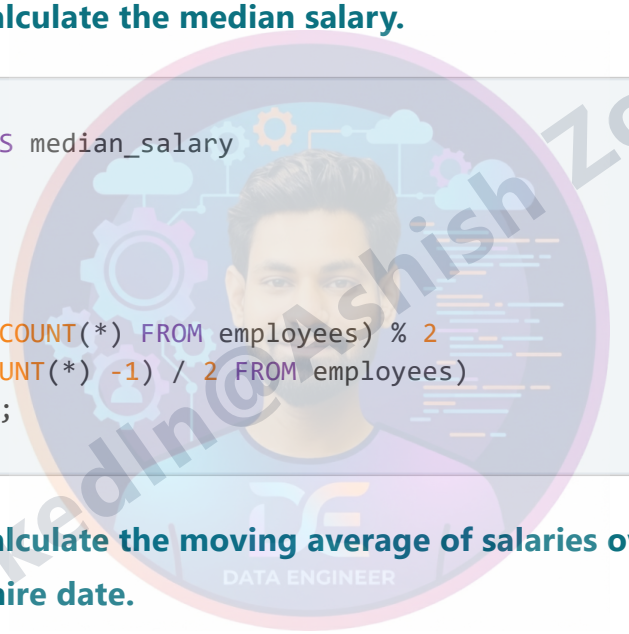
```sql
SELECT c.customer_id,
MIN(s.purchase_date) AS first_purchase,
MAX(s.purchase_date) AS last_purchase
FROM customers c
LEFT JOIN sales s ON c.customer_id = s.customer_id
GROUP BY c.customer_id;
```

140. Write a query to rank employees by salary with ties handled properly.

```sql
SELECT name, salary,
RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

141. Write a query to calculate the median salary.

```sql
SELECT AVG(salary) AS median_salary
FROM (
  SELECT salary
  FROM employees
  ORDER BY salary
  LIMIT 2 - (SELECT COUNT(*) FROM employees) % 2
  OFFSET (SELECT (COUNT(*) -1) / 2 FROM employees)
) AS median_subquery;
```



142. Write a query to calculate the moving average of salaries over the last 3 employees ordered by hire date.

```sql
SELECT name, hire_date, salary,
AVG(salary) OVER (ORDER BY hire_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
AS moving_avg_salary
FROM employees;
```

143. Write a query to calculate the difference between current row and previous row's salary.

```sql
SELECT name, salary,
salary - LAG(salary) OVER (ORDER BY id) AS salary_diff
FROM employees;
```

144. Write a query to calculate salary difference between employee and manager hire dates.

```
SELECT e.name AS employee, m.name AS manager,
EXTRACT(DAY FROM (e.hire_date) - (m.hire_date)) AS days_difference
FROM employees e
JOIN employees m ON e.manager_id = m.id;
```

145. Write a query to count employees in each department having more than 5 employees.

```
SELECT department_id, COUNT(*) AS num_employees
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

146. Write a query to find departments with no employees.

```
SELECT d.department_name
FROM departments d
LEFT JOIN employees e ON d.department_id = e.department_id
WHERE e.id IS NULL;
```

147. Write a query to list all departments and their employee counts, including departments with zero employees.

```
SELECT d.department_id, d.department_name, COUNT(e.id) AS employee_count
FROM departments d
LEFT JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;
```

148. Write a query to find employees who earn more than the average salary in their department but less than the company-wide average.

```
SELECT *
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department_id =
e.department_id)
AND salary < (SELECT AVG(salary) FROM employees);
```

149. Write a query to find employees with salary above the average salary of their department but below the company-wide average.

```
SELECT *
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department_id =
e.department_id)
AND salary < (SELECT AVG(salary) FROM employees);
```

150. Write a query to find duplicate rows based on multiple columns.

```
SELECT column1, column2, COUNT(*)
FROM table_name
GROUP BY column1, column2
HAVING COUNT(*) > 1;
```